

Exploration with Bayesian Techniques in Tetris

Alfie Lenton, University of Bath

8th September 2025

Contents

1	Introduction	3
2	Related Work	3
2.1	Foundational RL Principles	3
2.1.1	Markov Decision Process and the reinforcement learning setup	3
2.1.2	Action-value functions and Bellman Equations	4
2.1.3	TD Learning	5
2.1.4	DQN Model	5
2.2	Exploration-Exploitation Trade-off	7
2.3	Bayesian Principles in RL	8
2.3.1	Bayesian Methods	8
3	Methods	9
3.1	Bayesian Principles in Reinforcement Learning	9
3.2	Bayesian Exploration	9
3.3	Thompson Sampling	10
3.4	Upper Confidence Bound	12
3.5	Model: Bayesian DQN	14
3.5.1	Policy 1: Thompson Sampling of the weight posterior	15
3.5.2	Policy 2: Thompson sampling of the predictive distribution	16
3.5.3	Policy 3-4: Upper Confidence Bound	16
3.6	Complete Algorithm	17
4	Experiments	18
4.1	The CartPole environment	18
4.1.1	Policy 1 results	19
4.1.2	Policy 2 results	20
4.1.3	Policy 3 results	20
4.1.4	Policy 4 results	21
4.2	Tetris	21
4.2.1	Priority Buffer	22
4.2.2	Reward Shaping	22
4.2.3	Tetris results: Policy 1	23
4.2.4	Tetris results: Policy 2	23
4.2.5	Tetris results: Policy 3	23
4.2.6	Tetris results: Policy 4	24
4.2.7	Baseline Comparison	24
5	Further Work	25
5.1	Computation time	25
5.2	Uncertainty as a measure for exploration	25
6	Conclusion	26

1 Introduction

In the reinforcement-learning (RL) scenario, one of the most widely recognised and discussed dilemmas is the exploration-exploitation trade-off. In very large state spaces, it is difficult to find effective algorithms that explore state spaces efficiently so that an optimal policy is found within a learning time. Initial methods such as ϵ -greedy have shown good empirical results, however this algorithm can explore inefficiently as all sub-optimal actions are viewed as equally valuable when exploring. A more suitable algorithm would find a way to quantify the value in exploring particular actions and then take these values into consideration when choosing an action to execute.

In supervised and unsupervised learning problems, Bayesian approaches have shown more informative analysis than standard approaches. A Bayesian approach to RL can be a natural and effective way of exploring. Bayesian machine learning has recently shown promise in supervised and unsupervised learning problems as it works on the basis of gaining a more holistic view of observed data by obtaining distributions, rather than typical machine learning solutions which aim to provide a simple point estimate to predictions with no measure of uncertainty or structure to these predictions [9].

With a more informative model for a reinforcement learning problem, Bayesian-based models have been shown to perform at an impressive standard and to outperform models that do not take this broader-reaching approach on board, such as Double DQN and Duelling DQN. This paper aims to evaluate some of the ways in which a Bayesian approach to reinforcement learning can be both more informative about an environment and enable a policy of more efficient exploration.

To test the efficiency of these principled approaches to exploration, the environment used is Tetris. Tetris is a game where pieces descend a board and a player has to manoeuvre these pieces in order to create complete, horizontal lines across the board. The player has the option to either move the pieces left or right, move them down faster, rotate the pieces or do nothing i.e. let the pieces fall unaltered. The difficulty of this environment for an RL agent comes from the exploration required to discover positive rewards. As positive rewards are only observed when pieces are built to form a line, it requires a long sequence of actions to build these lines, which becomes difficult for an RL agent to observe. Therefore, principled exploration is essential for progress in this environment.

2 Related Work

2.1 Foundational RL Principles

2.1.1 Markov Decision Process and the reinforcement learning setup

The setup to the reinforcement learning problem is modelled as a Markov decision process (MDP). A Markov decision process over a horizon is an environment

consisting of a tuple $\langle \mathcal{X}, \mathcal{A}, T, R, \gamma \rangle$ in which the $\mathcal{X}$ is the state space, $\mathcal{A}$ is the action space, T is the transition function, R is the reward function and γ is the discount factor. At a time step t , the environment is in state $s_t \in \mathcal{X}$. The agent, under a policy $\pi(a_t|s_t)$, then chooses an action $a_t \in \mathcal{A}$ to execute. On doing so, the agent receives a reward based on the function $r_t = R_t(s_t, a_t)$ and the environment changes to a new state s_{t+1} under the transition function $T(s_{t+1}, r_t|s_t, a_t)$. The aim of the agent is to find a policy π that maximises the total discounted reward over the horizon. The expression for this is shown below:

$$\max_{\pi} \lim_{N \rightarrow \infty} \mathbb{E} \left[\sum_{t=0}^N \gamma^t r_t \right]$$

Reinforcement learning structures can be categorised into model-based and model-free agents. In the former category, data observed by the environment is used to try and model the environment's dynamics, that is the transition function T , and this model is used to derive an optimal policy. In the model-free case, it is the discounted reward for which a model is precisely approximated, ignoring the dynamics of the environment, and an optimal policy is chosen according to what the predicted optimal discounted reward is for each state and action pair. Both approaches to reinforcement learning have shown effective empirical results; there is evidence to show that with less data a model-based approach can be more effective, but whether one is better than another is highly disputed [DBLP:journals/corr/GhavamzadehMPT16]. In both of these categories, the exploration-exploitation tradeoff needs to be carefully considered for agents to be effective. In this paper, the focus is a model-free approach that uses Bayesian inference.

2.1.2 Action-value functions and Bellman Equations

One of the fundamental principles of Reinforcement Learning (RL) that will be utilised in this paper is action-value functions[13]. Action-value functions are a formulation of how valuable taking a certain action is, in a given state; it is these functions that a reinforcement learning agent seeks to accurately predict and build its policy on. The value of a state-action pair is stated to be the expected total discounted reward received, that is for a policy π and state-action pair (s, a) , the action-value function is:

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a]$$

where $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k}$ is the total discounted reward, $\gamma \in (0, 1]$ and R_t is the reward received at time t . The optimal action-value function is the action-value function that uses the optimal policy π^* , which is the policy that maximises the action-value function, that is:

$$q^*(s, a) = \max_{\pi} q_{\pi}(s, a)$$

These action-values are tuned and estimated over the experience the agent has with the environment.

From this formulation of action-value functions comes a fundamental property that can be exploited: that is the recursive nature of the action-value function. The recursive nature gives rise to the Bellman equation, the set of equations on which almost all RL models base their learning experience. The Bellman equations are derived as:

$$\begin{aligned}
q_\pi(s, a) &= \mathbb{E}_\pi[G_t | S_t = s, A_t = a] \\
&= \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k} | S_t = s, A_t = a\right] \\
&= \mathbb{E}_\pi\left[R_t + \gamma \sum_{k=0}^{\infty} \gamma^k R_{t+1+k} | S_t = s, A_t = a\right] \\
&= \mathbb{E}_\pi[R_t + \gamma G_{t+1} | S_t = s, A_t = a] \\
&= \mathbb{E}_\pi[R_t + \gamma q_\pi(S_{t+1}, A_{t+1}) | S_t = s, A_t = a]
\end{aligned} \tag{1}$$

This derivation shows that an action-value can be worked out with the action-value at the next state with the next expected action at that *next state*, along with the reward received from the current action-value pair and the discount factor γ . This allows for temporal difference learning.

2.1.3 TD Learning

Temporal Difference (TD) learning methods are an advancement of Monte Carlo return methods. Where Monte Carlo Methods have to wait for an entire episode to be played in order to make updates, TD learning methods, according to the equation above, make updates to the agent while an episode is being carried out. This method of learning updates towards '*bootstrap*' estimates. This means that the target value the updates are made towards are based on our estimate of the action-value function. The fact that TD learning can be implemented 'online' provides the clear advantage of TD methods as opposed to Monte Carlo methods, and makes it suitable for Tetris gameplay.

2.1.4 DQN Model

In original methods of RL, approximations of state values or state-action values were made with tabular methods. This means that each state or each state-action pair was listed and given an approximation as to its values. Tabular methods are effective for small or discrete action spaces but are impractical when these spaces are either larger or continuous. For the Tetris environment, while the action space remains small and discrete (only five distinct actions), the state space is the images outputted from the environment, which means that the state space is continuous and thus tabular methods become impractical for this purpose. With this being the case, function approximation is required [13].

Rather than explicitly listing state-action combinations, a function approximation uses a parameterised function that takes in as input the state-action pair and outputs the value of this pair. Throughout the process of the RL problem, the parameters of the function are adjusted so that the function approximation becomes more accurate to TD estimates of the true action-value.

As the Tetris environment’s state space is the images of the current state of the board, a suitable function approximation for this uses a convolutional neural network (CNN). Added to this, a common strategy for Tetris is to recognise similar patterns between the current block that is being moved and the shape and layout of the blocks that have previously been placed. For a reinforcement learning agent to be able to discover this strategy, it would need a function approximation that could recognise patterns and utilise computer vision methods; for these reasons a CNN also becomes a very practical choice [7].

One of the most widely known implementations, and on which the foundation of the model used in this paper is based, of an RL agent using a CNN as a function approximation is the Deep Q-Network (DQN) [6]. Included in the structure of the DQN is an experience replay buffer. An experience replay buffer is used to store transitions from experiences with the environment. When the agent is shown state s_t and performs a certain action a_t , it will receive a reward r_t and a next state s_{t+1} . These combinations (s_t, a_t, r_t, s_{t+1}) are stored in a dataset $\mathcal{D}$ known as the replay buffer. When making updates to the CNN, transitions can be sampled from the replay buffer, target values can be calculated using the Bellman equations and the CNN parameters can be updated with gradient descent, equivalent to typical supervised learning problems. The updates made are analogous to the TD learning process stated previously.

To help with convergence of the CNN parameters, a final structure is added: the separation of a target network (used to calculate target values) and an online network (the network to which TD learning updates are applied) [6]. The reason this final adjustment is made is that if the learning network is used to both calculate target values and choose the actions taken in the experiences, the updates made to the parameters will move towards a moving target and will result in slow convergence. Therefore, a target network is used to calculate the target values. This target network is identical in structure to the online network but does not change its parameters so that state-action pairs will give the same output over the learning process, and the convergence of the online network is faster. The target network’s parameters are updated to equal the parameters of the online network after a set number of steps. The full DQN algorithm is stated below and will be very similar to the foundational model used in this paper:

Algorithm 1 Double DQN algorithm

```
Initialise replay buffer  $\mathcal{D}$  with capacity  $N$ 
Initialise action-value functions  $Q_\theta$  and  $Q_{\theta^{\text{target}}}$  with parameters  $\theta$  and  $\theta^{\text{target}}$ 
Set  $T^{\text{target}}$ 
 $\text{count} \leftarrow 0$ 
for episodes = 1, ...,  $M$  do
  Observe initial state  $s_1$ 
  while episode not done do
    Select action  $a_t$  according to policy  $\pi(a_t|s_t)$ 
    Execute action  $a_t$  and observe  $r_t, s_{t+1}$ 
    Store transition  $(s_t, a_t, r_t, s_{t+1})$  in replay buffer  $\mathcal{D}$ 
    Sample random mini-batch of transitions  $(s_j, a_j, r_j, s_{j+1})$  from  $\mathcal{D}$ 
    Set  $y_j = \begin{cases} r_j & \text{if } s_{t+1} \text{ is terminal} \\ r_j + \gamma Q_{\theta^{\text{target}}}(s_{j+1}, \mathbb{E}[\pi(a|s_{j+1})]) & \text{otherwise} \end{cases}$ 
    Perform gradient-descent step on  $(y_j - Q_\theta(s_j, a_j))^2$  on parameters  $\theta$ 
     $\text{count} \leftarrow \text{count} + 1$ 
    if  $\text{count} \bmod T^{\text{target}} = 0$  then
       $\theta^{\text{target}} \leftarrow \theta$ 
```

2.2 Exploration-Exploitation Trade-off

In the Tetris environment, the problem of sparse rewards is a challenging one to overcome. This is the problem of how the majority of actions chosen by the agent will not reap rewards until it reaches much further distant time steps. If the agent does not see rewards soon after a particular action is chosen, it becomes difficult to recognise certain state-action pairs or combinations of state-action pairs as beneficial for the agent. Whilst it is logical for an agent to choose actions that maximise the action value, in sparse reward problems, such as Tetris, this may not be feasible [5]. The actions that maximise this action value are called *exploitative*. In this case, *explorative* policies, where actions are chosen not necessarily according to their value, may be more effective.

The most obvious approach to building an explorative policy, is a policy that chooses actions randomly regardless of the state. Whilst this explores effectively, there is no exploitative quality to this, so the agent will struggle to receive maximum rewards. A step towards a more exploitative policy, but one that maintains a random exploration element, is the ϵ -greedy policy [6]. For a time decaying parameter $\epsilon \in (0, 1)$, the ϵ -greedy policy chooses the action a such that $a = \underset{a' \in \mathcal{A}}{\operatorname{argmax}} Q(s, a')$ with probability $1 - \epsilon$ and chooses a random action $a \in \mathcal{A}$ with probability ϵ . This means that with initial high ϵ , the agent will choose random actions more often and hence will explore more. But after a significant number of actions are executed, the hope is that the agent will have explored enough to find a suitable policy and will now have a low ϵ value and can now use a more optimal policy to exploit.

Whilst this exploration strategy has shown impressive results in many in-

stances of Atari gameplay, the undirected nature of this type of exploration can lead to wasted instances of exploration. Undirected exploration means that when the agent aims to explore, it sees each choice of action as being equally valuable [1]; this is formulated in this policy in how, when exploring, each action has an equal chance of being chosen.

Another example of exploration is reward shaping. With reward shaping, artificial rewards are given to the agent depending on certain actions and the next state [11]. For example, some reward shaping in Tetris may include positive rewards for actions that do not terminate an episode and negative rewards for actions that do, hopefully guiding the agent to choose actions that extend the length of the episode. Another example would be to give rewards to the agent based on the number of holes there are in the image of the next state [12]. Holes in the Tetris board are empty squares of the Tetris board in which the square directly above it is not empty. Holes impede progress in Tetris as the hole can only be filled once the line above is cleared, so giving the agent information about this in the form of artificial rewards may help the agent to build a strategy based on reducing the number of holes on the board. However, there are problems with reward shaping as the rewards received depend heavily on the designer and may not have the intended results one would like to see of the agent. Also, in more general environments, information about the environment, like that of holes for Tetris, may not be readily available so it may not be as applicable.

While the above approaches towards exploration are sensible and even show impressive empirical results they are either undirected (in the case of ϵ -greedy) or require detailed prior knowledge of the environment (in the case of reward shaping) and are therefore not generally applicable. In this paper, I aim to experiment with methods of exploration that are both directed and of general use. With this in mind, more information is needed about an environment during learning than simply a reward and next state. This is where the application of Bayesian principles can be of help.

2.3 Bayesian Principles in RL

2.3.1 Bayesian Methods

Typical machine learning solutions, specifically supervised and unsupervised, have approached prediction problems with vast, complex architectures, such as multilayer perceptrons, which are then, given observed data, adjusted and finely tuned with algorithms like gradient descent to give accurate predictions for new data. Often in machine learning problems, a probabilistic approach based on Bayes theorem is both a natural and effective basis to make predictions: the approach proposes a way to model the data, observes real data of the phenomenon and then adjusts its model with Bayesian inference. Typically for a particular model M and observed data D , the Bayesian inference formula is such:

$$P(M|D) = \frac{P(D|M)P(M)}{P(D)}$$

Whilst in this example, it shows that a model is updated according to the observed data, there can be other applications for Bayesian inference, such as updating specific parameters of the model based on observed data or even making new predictions based on observed data.

Machine learning methods, such as neural networks, have shown impressive results, however there are downfalls to a more "black-box" approach. Firstly, it is hard to impart any prior knowledge about a scenario onto a model of this kind, whereas a Bayesian approach has a prior knowledge term written into it. Secondly, the black-box nature of these structures make interpreting parameters inside the model quite difficult when these parameters could have real-world physical interpretations, about which it would be helpful to learn. Finally, and this is the point on which this paper will focus, black-box models make only point estimates for predictions [9]. While some black-box models can give uncertainty estimates to these predictions, a Bayesian framework gives a more holistic view of the data and a natural and more statistically sound uncertainty estimate alongside predictions. It is these values that can enable an effective exploration strategy.

3 Methods

3.1 Bayesian Principles in Reinforcement Learning

With a more holistic view of the action-value function predictions, a more directed approach can be taken on exploration. In RL the difficulty comes in obtaining a good data collection strategy i.e. the policy used. The policy needs to be able to efficiently explore the policy space but also needs to focus data collection which can learn a policy that yields the best outcomes. The use of Bayesian inference in RL provides a sensible, mathematically principled way to approach this problem. Bayesian inference allows the designer to provide prior information about the system in the form of a probability distribution and for this information to be updated and fine-tuned when observations are made about the environment.

As with standard Bayesian machine learning, one of the advantages of a Bayesian approach is a natural facilitation of regularisation [9]. With a prior distribution assumed on the model of the action-value function, this reduces the susceptibility of the model to be steered away from accuracy by a few outlying data points. Another advantage of the Bayesian approach is that a posterior distribution updated by observations from the environment can be a natural hallmark on which to base a policy.

3.2 Bayesian Exploration

Traditional exploration methods such as ϵ -greedy have shown promise in RL problems, however they rely on pure randomness for exploration. This tactic for exploring can be inefficient as it explores all actions with equal chance when

this is unlikely to be sensible as exploring some actions may be potentially more valuable than others. Also, the ϵ -greedy tactic decays the ϵ parameter over time, allowing for more exploration in the beginning, when there is little knowledge about the environment, and for more exploitation towards the end of the training time, by this time more knowledge of the environment will have been gained and a more efficient exploitative policy can be utilised. However, the time-decay of this parameter that defines exploration for the agent comes with an assumption. The assumption is that the knowledge gained from interaction with the environment will, in turn, form a policy that will become more accurate and efficient at a rate similar to that which the ϵ parameter decays. In complex environments, this is quite a bold assumption as interaction with the environment may not necessarily mean a broader knowledge gain of the environment. This means that the exploration parameter may decay too early before an effective exploitative policy can be established, so when this policy is then used to start exploiting, it may not be exploiting with a truly optimal policy.

These exploration methods that rely on pure randomness for exploration are sensible in the traditional approaches of RL as the agent, in the model-free setting, obtains just a point estimate as to what the value is of a particular state-action pair. As this is the only value on which an agent can form the basis of a policy, a policy based purely on this estimate would be only exploitative. Given little else to work with, including randomness is a sensible way to gain an explorative element of the policy. Bayesian inference gives the agent not only a point estimate of a state-action pair but also an uncertainty estimate of the value of a state-action pair from which a distribution can be formed over the random variable of the value of a state-action pair [1]. This distribution gives the agent a method for exploration that is directly inferred from previous interactions made with the environment. Not only is this a more principled method for exploration, as it encodes knowledge gained from the environment into this distribution, but it is also adaptive as to the knowledge gained from the environment. This avoids the assumption that knowledge of how to exploit the environment will be gained in a stable way, and allows the agent to vary its exploration based on the uncertainty of the exploitative value of taking certain actions, allowing for the fact that this uncertainty may vary over training time. As will be shown later, with the Bayesian approach it is still a fair assumption that exploration can decrease over time, however this assumption is encapsulated in the uncertainty decreasing rather than the ϵ parameter decreasing, which is unrelated to the environment.

3.3 Thompson Sampling

The Thompson sampling method is considered one of the most natural Bayesian approaches to the exploitation-exploration problem [DBLP:journals/corr/GhavamzadehMPT16]. Thompson sampling utilises posterior distributions that model the environment to select an optimal action. A general Thompson sampling algorithm is below [3]:

Algorithm 2 Thompson Sampling

algorithmic

Set P_{prior} over θ

$P_{post} \leftarrow P_{prior}$

Initialise episode with state s_0

while episode not done **do**

for $t = 0, 1, \dots$ **do**

 Sample $\hat{\theta}$ from P_{post}

 Select action $a_t = \arg \max_{a' \in \mathcal{A}} \mathbb{E}_{P_{\hat{\theta}}(G_t|a')} [G_t]$

 Observe reward r_t and next state s_{t+1}

 Update P_{post}

The Thompson sampling algorithm can be seen to have a logical approach to exploration. For parameter values that have more uncertainty, parameter values that could lead to a different optimal action being chosen have a higher chance of being sampled. As more observations of less certain actions occur, the uncertainty in those actions decreases and then the likelihood of that action being chosen in the future is based more on its exploitative benefit rather than the benefit from exploring this action. This method for exploration is an approach that uses the observations made from the environment to shape how this policy explores and relies on the uncertainty, a logical statistic, to define how likely explorative actions are chosen over exploitative actions.

Thompson sampling shows great empirical results in a simple environment such as the multi-arm bandit problem. The parameters θ sampled in this case were the predicted probabilities of receiving a reward given the action a ; in the multi-arm bandit case, actions translate to choosing which arm to pull. As these parameters are updated with Bayesian inference, an ideal choice for a prior belief over the parameters is a Beta distribution:

$$\theta \sim \text{Beta}(\alpha, \beta)$$

This choice is ideal as, in the multi-armed bandit case, the rewards of the environment are Bernoulli distributed where the probability of receiving a positive reward for taking action a_i is θ_i . The Beta distribution is conjugate to the Bernoulli. With a beta prior belief on the θ parameters, updating the parameters to a posterior belief in this case is very easy. The updated posterior belief becomes:

$$\theta_i | \mathbf{X} \sim \text{Beta}(\alpha + \text{no. successes of arm } i, \beta + \text{no. failures of arm } i)$$

As actions are taken, the posterior beliefs of the θ_i , the parameters for each arm, are updated. Then to take a new action, parameter values are sampled

from their posterior distributions, and the action that yields the maximum reward, with respect to the sampled parameters, is chosen. In the multi-arm bandit case, this would be the highest sampled parameter. This policy means that each arm is initialised to have an unknown probability of yielding a reward, so all arms are equally likely to be chosen. As observations are made, the beliefs about the parameters are updated. The more frequently successful arms will cause their respective parameters' expected values to shift towards 1 while more frequently failing arms will shift their parameters' expected values towards zero.

So in the initial stages, exploration dominates when little is known about the environment and the parameters dictating it. Towards the end of training, this shifts to more exploitative actions being taken when a more accurate belief over the parameters dominates and the more profitable arms have a higher chance of being chosen.

In simple cases like this, the parameter θ to use for Thompson sampling is an easy and obvious choice. In more complex environments, such as Tetris, this choice is made harder.

3.4 Upper Confidence Bound

The upper confidence bound uses the uncertainty of the rewards received to avoid overlooking what could be the best action to take in a given state. The upper confidence bound works on the principle of choosing an action, not only based on its expected discount reward, but based on its uncertainty regarding the discounted reward. Where Thompson sampling uses the uncertainty of this value to gain a more stochastic policy (taking samples from a distribution), upper confidence bound uses the uncertainty more directly and more deterministically.

With Bayesian inference, a distribution is established over the potential Q-values. As stated previously, building a distribution over these values rather than taking point estimates allows for the environment to have stochasticity and allows for the fact that rewards may not be consistent or easily predictable. However, with a distribution, it is possible to establish what values are likely, for example, with a confidence interval. A range of values can be established within which the true q-value has a high probability of landing. With this, an expected value can be found but also an upper bound can be found and utilised. The intuition behind basing action selection on an upper bound rather than the expected value is that it enables exploration by not necessarily choosing the expected best action but the action that has the potential to be the best action. Once this action is taken, the value of it can be learned with future network and the posterior is updated accordingly. As with Thompson sampling, this intuition was first illustrated in the multi-armed bandit problem.

As stated previously, the multi-armed bandit uses a beta prior to update the belief of the set of parameters θ which model the probability of pulling arm i being a success. The general formulation for the upper confidence bound policy for arm i is [4]:

$$\text{UCB}(i) = \mu_i + \beta_t \sigma_i^2$$

where μ_i, σ_i^2 are the expected reward and uncertainty of arm i and β_t is the exploration-weighting parameter that is scheduled over time. The way in which uncertainty is measured in the multi-armed bandit case can vary from model to model. This is one formulation:

$$\beta_t \sigma_i^2 = \sqrt{\frac{2 \ln t}{t_i}}$$

where t is the current timestep and t_i is the number of times arm i has been pulled. This formulation characterises uncertainty as being related to the number of times an action is taken. The less often an arm has been pulled, the more uncertainty is attributed to it. This optimistic upper confidence lowers the risk of an arm being overlooked due to unlucky realisations of that arm. Whilst this is a sensible formulation in the multi-arm bandit case, this does not extend to general MDPs. In the paper [4] the upper confidence bound is extended to a Bayesian setting. Rather than a measurement of uncertainty being taken that is unrelated to the actual distribution of the parameters, this upper confidence bound is found by taking the value at the top end of an upper confidence bound. This upper confidence bound takes high quantile values as the sampled value of a particular action. The quantile found is formulated below:

$$q_j(t) = Q\left(1 - \frac{1}{t \log(n)^c}, \lambda_j^{t-1}\right)$$

where t is the timestep, n is the total number of timesteps, j is the arm chosen, c is an exploration control parameter and λ_j^{t-1} are the posterior parameters updated to timestep $t - 1$ of arm j . With this calculation, the value of $q_j(t)$ is such that $\mathbb{P}(Q \leq q_j(t)) = 1 - \frac{1}{t \log(n)^c}$, so $q_j(t)$ of the confidence level at timestep t . Larger c means that the confidence interval is wider, so actions that may have a lower chance of performing well are given a higher chance of being selected. As time goes on, while the interval does get wider, the posterior belief should have less uncertainty and therefore the upper quantile will become smaller for actions with less uncertainty attributed to them. The image below helps to illustrate the principle of upper confidence bound:

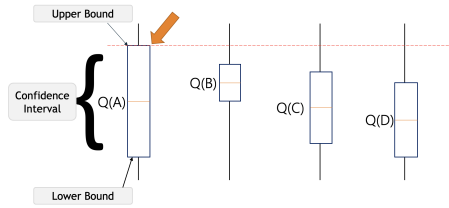


Figure 1: Demonstration of the principle of UCB: [2]

As you can see above, while action A has a lower expected value than action B , action A has a higher upper confidence, or a higher potential value. The upper confidence bound policy selects action A for this reason. With the informative statistics that Bayesian inference provides, a sensible upper confidence bound can be found for each action and actions can be selected according to their potential.

3.5 Model: Bayesian DQN

The Bayesian DQN is an adaptation of the typical Double DQN model proposed by [6]. In the Tetris environment, the state space is the images received of the current state of the board of the game after processing. The frames are received in stacks of four and the images are resized to 84 by 40.¹ The complete neural network is similar to the DQN model; it consists of three convolutional layers and a final fully connected layer. The first convolutional layer has 32 kernels of size 8 by 8 with a stride of 4. The second convolutional layer outputs 64 channels with kernels of size 4 by 4 and a stride 2. Lastly, the final convolutional layer has 64 output channels again with a kernel size of 3 by 3 and a stride of 1. Each of the convolutional layers has a ReLU activation. With the output of the final convolutional layer being flattened, the final fully connected layer, with no activation, outputs a feature extraction of dimension 512. This feature extraction is passed through the Bayesian layer and serves as the input data on which Bayesian linear regression is performed.

The weights of the Bayesian layer provide the basis on which Bayesian inference is performed [1]. To be able to utilise Bayesian inference, a prior belief and a likelihood have to be defined over these weights. For Bayesian linear regression models, a common choice is conjugates of each other. This makes the posterior distribution over the weights analytically tractable. With this being analytically tractable, a predictive distribution can also be calculated. In this case, Gaussian priors and likelihoods are used as these are conjugates and form a Gaussian posterior with statistics that can be calculated given data.

The Q-functions of a given (s, a) pair are modelled as the dot product of the feature extraction of the state, denoted $\phi_\theta(s)$ and the weights of the given action, denoted w_a , that is $Q(s, a) = \phi_\theta(s) \cdot w_a$. For a given observation of the environment (s, a, r, s') , calculations of the target values y follow from the calculations made for targets in the Double DQN case. With a fixed target network *and* a fixed set of weights of the Bayesian layer, the target values are calculated as:

$$y = r + \gamma \phi_{\theta^{target}}(s') \cdot w_{a'}^{target}$$

where a' is the action selected at s' according to the current policy. With these target values, we can approximate a posterior distribution of the weights $\{w_a\}_{a \in \mathcal{A}}$ from the prior and likelihood distributions. The weights $\{w_a\}_{a \in \mathcal{A}}$ are

¹in [6] images are resized to 84 by 84, however for Tetris the images are cropped as a lot of space on the images consists only of the scoreboard which is not relevant to progress for the agent.

drawn from the Gaussian prior distribution $\mathcal{N}(0, \sigma_w^2)$ and the target values are modelled to have Gaussian noise:

$$y = \phi_\theta(s) \cdot w_a + \epsilon$$

where $\epsilon \sim \mathcal{N}(0, \sigma_n^2)$ is i.i.d noise, and so the likelihood is $y|s, a, w_a \sim \mathcal{N}(\phi(s) \cdot w_a, \sigma_n^2)$. Using Gaussian priors and likelihoods gives a normalising factor $\int_{w_a} \mathbb{P}(y|w_a) \mathbb{P}(w_a) dw_a$ that has a closed form and that is analytically tractable.

For a given batch sample of the experience replay buffer $\mathcal{B} = \{s_\tau, a_\tau, r_\tau, s'_\tau\}_{\tau=1}^B$ where B is the size of the batch, the sample can be split into disjoint datasets according to the actions where $\mathcal{B}_a = \{(s_\tau, a_\tau, r_\tau, s'_\tau) | a_\tau = a\}$, with size B_a . For action a and partitioned dataset $\mathcal{B}_a$, a matrix $\Phi_a \in \mathbb{R}^{d \times B_a}$ where the columns of the matrix are the feature extractions $\{\phi(s_i)\}_{s_i \in \mathcal{B}_a}$ and d is the size of the feature extractions. A target vector $\mathbf{y}_a$ is also calculated by

$$[\mathbf{y}_a]_i = \begin{cases} r_i & \text{if } s'_i \text{ is a terminal state} \\ r_i + \gamma \phi(s_i) \cdot w_{a'} & \text{otherwise} \end{cases}$$

With this matrix and target vector constructed, the posterior distribution of w_a is such [1]:

$$w_a \sim \left(\frac{1}{\sigma_n^2} \Xi_a \Phi_a \mathbf{y}_a, \Xi_a \right) \text{ where } \Xi_a = \left(\frac{1}{\sigma_n^2} \Phi_a \Phi_a^\top + \frac{1}{\sigma_w^2} \mathbb{I} \right)^{-1}$$

where $\mathbb{I}$ is the identity matrix. From this, the predictive distribution of Q -values can be shown to be:

$$Q(s, a) | \mathcal{B}_a \sim \mathcal{N} \left(\frac{1}{\sigma_n^2} \phi_\theta(s) \cdot (\Xi_a \Phi_a \mathbf{y}_a), \phi_\theta(s) \cdot (\Xi_a \phi_\theta(s)) \right)$$

With these more informative statistics of the model of the Q -values, more options are given for policies that explore efficiently.

3.5.1 Policy 1: Thompson Sampling of the weight posterior

In the paper in which this model is proposed, [1] Thompson sampling is used over the updated posterior weight distribution. The Thompson sampling here follows the general algorithm of Thompson sampling in the multi-armed bandit environment stated before. Weights w_a are sampled from the posterior distributions (in the initial case, the prior distribution) and the action chosen is the action that maximises the predicted Q -value according to these sampled weights. This is formulated below:

$$a = \pi(s|w_{a'}) = \arg \max_{a' \in \mathcal{A}} w_{a'} \cdot \phi(s)$$

3.5.2 Policy 2: Thompson sampling of the predictive distribution

The previous calculations provide us not only with a posterior weight distribution, but also a predictive distribution over the Q -values. This paper explores the effectiveness of a Thompson sampling principle over the predictive distribution as well as the posterior weight distribution. In [1], there is doubt as to whether sampling over this distribution will be effective as the target data $\mathbf{y}_a$ that this distribution aims to model has inherent uncertainty in it that is not accounted for. The target vector is calculated based on bootstrapped estimates of future Q -values. In posterior calculations, it is assumed that this target vector provides the true discounted returns. However, given the promising empirical results shown by sampling from the weight distribution (which samples from a distribution calculated with the same target vector), it is not unreasonable to hypothesise that sampling over the predictive distribution could show similar results. Therefore, for a given state, this policy calculates the predictive distribution for each action, samples q -values from each distribution, and chooses the action that yielded the maximum sampled value. The formula for this is as follows:

$$a = \pi(s) = \arg \max_{a' \in \mathcal{A}} q_{a'}$$

such that $q_{a'}$ is sampled from $\mathcal{N}(\phi(s) \cdot \mathbb{E}[w_{a'}], \phi(s) \cdot (\Xi_{a'} \phi(s)))$

3.5.3 Policy 3-4: Upper Confidence Bound

The final policy tested in this paper is an adaptation of Upper Confidence Bound using the predictive statistics. As the principle of the upper confidence bound is to avoid overlooking the potential best action in favour of an action that has a maximal prediction, this policy uses the uncertainty in the predictive distribution to define this upper confidence bound. In the multi-armed bandit case, the uncertainty of an action was defined by the infrequency of that action being taken. With these posterior distributions we have more robustly defined uncertainties for each action. This policy also utilises an upper bound co-efficient parameter β which is scheduled over training time to reduce the impact of the uncertainty on action selection as time goes on and more accurate posteriors are established. This β parameter can be likened to the ϵ parameter in an ϵ -greedy policy. Where this β parameter differs from ϵ though is that in ϵ -greedy, ϵ is the only way in which exploration is defined in this policy, with no relation to the observations made regarding the environment. Whilst β does have an effect on exploration, the dominating and adaptive component to exploration is the variance (or standard deviation) of the predictive distribution. The complete policy is the sum of the expected Q -value and the uncertainty of the Q -value with the uncertainty weighted by β . This formula is shown below:

$$a = \pi(s) = \arg \max_{a' \in \mathcal{A}} \phi(s) \cdot \mathbb{E}[w_{a'}] + \beta \text{Var}(w_{a'})$$

As is shown later, a strong deterministic policy like this, that is one that chooses a hard maximum argument for an action and doesn't include stochasticity like Thompson sampling, can underperform in complex environments like Tetris. Therefore, a stochastic version of this policy is trialled as well where, for each step, an action is sampled from the below distribution:

$$\pi(a|s) = \underset{a' \in \mathcal{A}}{\text{softmax}}(\phi(s) \cdot \mathbb{E}[w_{a'}] + \beta \text{Var}(w_{a'}))$$

3.6 Complete Algorithm

Algorithm 3 General Bayesian DQN Algorithm

```

Initialise parameters  $\theta, \theta_{target}, W, W_{target}$  and  $Cov$ 
Initialise replay buffer and set  $counter = 0$ 
Select policy  $\pi(s)$ 
for episode=1 to T do
  Initialise  $s_0$  to initial state environment
  for t to the end of episode do
    if  $\pi(s)$  is policy 1 and  $count \bmod T_{sample} = 0$  then
      sample  $W \sim \mathcal{N}(W_{target}, Cov)$ 
    end if
    Select action  $a_t = \pi(s)$  ( $\pi(s|W)$  if  $\pi$  is policy 1)
    Execute  $a_t$  and observe  $r_t, s_{t+1}$ 
    Sample a random minibatch of transitions  $(s_\tau, a_\tau, r_\tau, s_{\tau+1})$  from replay
    buffer
    Update  $\theta$  with gradient descent and MSE loss
    if  $count \bmod T^{target} = 0$  then
       $\theta_{target} \leftarrow \theta$ 
    end if
    if  $count \bmod T^{Bayestarget} = 0$  then
      Update posteriors  $W_{target}$  and  $Cov$ 
    end if
    if  $\pi(s)$  is policy 3 and  $count \bmod \beta_{update} = 0$  then
      Schedule  $\beta$ 
    end if
     $count = count + 1$ 

```

The overall outline of the algorithms remain similar across all policies; on sampling a mini-batch from the replay buffer, Φ_a feature matrices and target vectors $\mathbf{y}_a$ are calculated and used to calculate mean-squared error loss and the online network is updated with gradient descent. The target vectors for each policy differ by how the actions at the next states are selected. The algorithms also differ by policy in terms of the level of exploration that occurs. For policy 1, the randomness is dictated by the parameter T_{sample} . For every T_{sample} steps taken, a new set of weights W is sampled from the current posterior distribution

and going forward it is this set of weights that is used to calculate Q -values and on which to base actions. For policy 2, exploration relies only on the posterior distribution, so rates of exploration only depend on the parameter $T^{Bayes\ target}$ which is how often the posterior distribution is updated. This parameter is common across all policies, however, as this parameter is the only parameter that dictates the level of exploration for policy 2, varying this parameter across the duration of training may be sensible for this policy. Finally, the uncommon and exploration-dicating parameter for policy 3 and 4 is $\beta \in [0, 1]$. For every β_{update} steps, the β value is scheduled down towards zero to reduce the impact of uncertainty in selecting actions.

4 Experiments

Tetris is a complex environment in which it is often hard to attain sustained, improved performance. Before observing the performance of these policies on Tetris, the policies were tested in a simpler environment to ensure efficiency and reliability in performance. This environment was the Cartpole environment.

4.1 The CartPole environment

The CartPole environment is a simple game for a reinforcement learning model to understand. In this environment, an agent takes control of a cart. In this cart, a pole stands upright initially and tilts left and right. The aim of the agent is to learn how to keep the pole upright by moving the cart left and right. This environment is illustrated below:

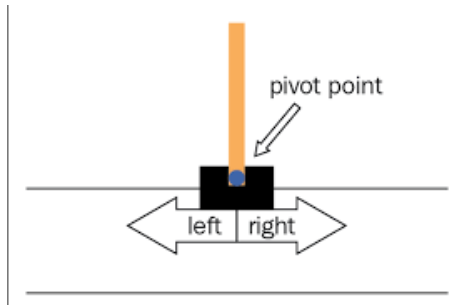


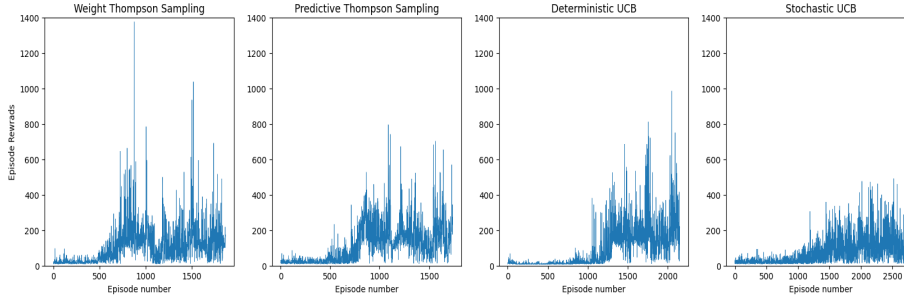
Figure 2: [14]

One of the reasons the CartPole environment is a good choice for ensuring the efficacy of these policies is that the state space dimension is much smaller than the Tetris state space, reducing compute time and giving the agent fewer variables to consider when choosing actions. The CartPole state space has four dimensions. The state consists of measurements of the position of the cart, the velocity of the cart, the angle of the pole and the angular velocity of the pole. An episode of the cartpole terminates if the cart moves outside of a particular

range or the pole falls outside of a particular angular range. The agent receives a reward of 1.0 for every timestep, meaning a high-performing agent will be able to keep the pole upright and the cart within a certain range for as long as possible.

Due to the change in the state space, the input layer of the Q-network needs to be modified. Rather than a dense CNN, the network used is a simple multi-layer perceptron network. It has an input layer of 4, then two hidden layers of 128 and 256 and an output layer of 512, all with biases. All layers, except the final output, have a ReLU activation, which then feeds into the Bayesian layer with 512 weights. It was found that with a ReLU activation on the final layer, the Bayesian updates were performed on data that had an over-saturation of zeros. Over time, this made all of the weights tend to zero and made the approximations ineffective.

As this environment was used to evaluate the initial effectiveness of the models, each model was trained over 200,000 steps. With this short training period, learning every step - rather than learning once a step-period had passed - did not incur much computational cost, so to see the effectiveness of the models, the networks learned every step, whereas in Tetris this was increased to every four steps. The Bayesian layer was also updated each time the network was learned. This general outline was common across all models with slight changes made depending on the requirements of each policy. As the positive rewards for CartPole are dense, the training curves below have the episode numbers plotted against the episode rewards for each agent:



4.1.1 Policy 1 results

The performance of Thompson sampling over the weights can be observed to have an improvement over time. At around 500 episodes, this policy has a moderately steep increase in performance at around 500 episodes and even achieves the best performance across all policies at episode 874, which is a total reward of 1,375 (translating to the pole being able to stay upright for 1,375 timesteps). However, at this point, the performance flatlines and maintains rewards of about 200, before decreasing in performance at around episode 1,200 where the average reward decreases to about 100. From this point on, performance does moderately increase again but doesn't consistently pass episode rewards of 200 again

over the training loop. The results show that the exploration tactic used here can explore the state space efficiently, but perhaps over-explore. This means that within the training time, the model explores enough to establish a policy that improves performance, but does not establish an exploitative policy, where uncertainty is reduced enough so that sampled weights can be a closer estimate of the true expected weights.

4.1.2 Policy 2 results

Thompson sampling over the predictive distribution shows a similar trend to the first policy. Policy 2 starts to show an improvement in performance slightly later than before, showing at around 800 episodes. Policy 2 exhibits a steeper and more steady initial increase than the first policy and steadily increases to averaging rewards of 300 before decreasing again to consistent rewards of around 150-200. After this, it seems to decrease further to rewards below the level of 100 before increasing again to rewards of around 300 and finishing the training loop with rewards of around 200. Whilst both the Thompson sampling methods show fluctuations in performance, both demonstrate an ability to improve overall performance in the CartPole environment. Given the shortness of the training loop (only 200,000 steps) these fluctuations may have been shown to level out over a longer training period as more time is allowed for the uncertainty in the two distributions to decrease.

4.1.3 Policy 3 results

Policy 3 required more fine-tuning to show good performance as the exploratory nature of the policy had more parameters guiding it, namely, the time-decay weight parameter β_t . As stated previously, β_t defined how much weight the uncertainty of an action in a particular state is given in selecting that action. This parameter was decayed over time so that uncertainty had less importance in the selection actions, making the policy drift from being more explorative to being more exploitative. After tests run with different kinds of decay, an exponential decay was chosen. The decay formula is shown below:

$$\beta(t) = \max(0.5, 3e^{-\frac{t}{d}})$$

where d is the decay rate that varies depending on the size of the training loop. Calibrating for d means choosing a d that β_t decays to 0.5 in an appropriate amount of time over the training loop. The decay rate dictates the proportion of training time that the policy spends exploring and exploiting. For the results shown above, a decay rate was chosen so that $\beta_t = 0.5$ when $t \approx 120,000$, that is $d = 67,000$. A minimum bound of 0.5 was also set on β_t to ensure that, even in the exploitative stages, the policy remains explorative, to an extent.

Policy 3 takes a much longer time to start showing signs of improved performance than Thompson sampling methods, only showing significant rewards at around episode 1,200. However, at this point, this policy shows the sharpest

increase of all policies trialled and increases to consistent rewards of around 200. Policy 3 consistently achieves rewards around this mark until around episode 1,750 where it decreases to rewards of about 100 before increasing again at episode 200 to rewards around 300 for the remainder of the training time. Comparing this performance to the performance of the two Thompson sampling policies, it can be seen that the performance is more consistent for this policy and shows a steady increase over time, ending with higher achieving episodes than both the first two policies. This observation can be put down to the fact that whilst this policy does have an element of adaptivity to its exploration, it is not as stochastic as Thompson sampling; this version of upper confidence bound has a deterministic structure and a parameter that defines harshly how much weight is given to exploration. Where this deterministic upper confidence bound explicitly defines a shift from exploration to exploitation, Thompson sampling allows the policy to shift the importance of the two sides of the trade-off dynamically over training time. For a short training period, it can be clear to theorise why a more deterministic policy will exhibit better performance over a stochastic policy. However, over a longer training period, the more thorough exploration of the state space by Thompson sampling could reap benefits over the harsh definition of exploration by upper confidence bound.

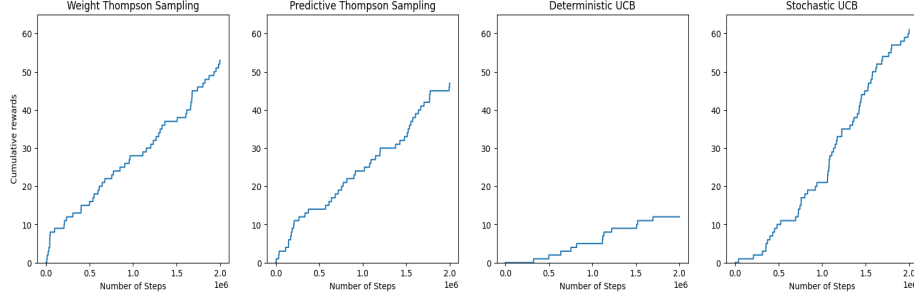
4.1.4 Policy 4 results

The stochastic version of the upper confidence bound exhibited the same decay parameter and decay rate used in the deterministic version. The only difference was that actions were sampled from a distribution rather than a maximal action selected. From about 1,000 episodes, this policy did show a steady improvement in performance over the whole training loop. However, it was the shallowest improvement in performance exhibited by any of the policies. This observation points to the fact that this policy explores on two fronts: the uncertainty of actions is taken into account, and actions are selected from a distribution rather than chosen according to their value. The lack of performance in this simple environment can be attributed to the added tier of exploration embedded within the policy. This further exploration means that the policy did not stabilise enough to start exploiting within the given training time. Although this policy showed limited performance in this simple environment, a stochastic version of UCB may be a more ideal choice than the deterministic version in an environment with a more complex state space and more sparse rewards, such as Tetris.

4.2 Tetris

For reinforcement learning models, Tetris’s main difficulty comes from sufficient exploration of the state space and learning how rewards are gained when they are sparse. This paper ultimately tries to evaluate how a more principled exploration tactic can perform in such an environment. However, given the sparseness of rewards and the difficulty of the environment, some slight adaptations to the

models were trialled. These adaptations are made in an effort to make more informative, denser rewards and so that the networks can learn optimal actions faster. The adaptations tested are a priority buffer and reward shaping.



4.2.1 Priority Buffer

The priority buffer is an adaptation made on the replay buffer. As rewards are sparse within Tetris, the replay buffer will not have many tuples stored where the rewards are positive, meaning that it will not often learn from data samples in which the action chosen received a reward as a direct result. The priority buffer means the likelihood of a sampled batch containing samples with positive rewards is increased [10]. The priority buffer stores samples with positive rewards in the regular replay buffer but also in a priority buffer which contains only significant data samples. Then, when a batch is sampled, the total batch contains samples from the regular replay buffer and also a proportion of samples from the priority buffer, where the maximum ratio these priority samples take of the total batch is set. In this case, the proportion set was 0.25.

However, after testing, this adaptation was shown to have no significant enhancement in performance. This may be because, in the early stages where some positive rewards are seen, these few data samples are repeatedly used for learning the network. However, with a relatively small batch size and little diversity of samples with positive rewards, the network may be overfitting to the few positive instances observed in the early stages, making it unable to generalise later on. For this reason, the priority buffer is not used in the final results.

4.2.2 Reward Shaping

Reward shaping transforms the raw rewards emitted from the environment so that the rewards observed by the agent have more information encoded. The raw rewards from Tetris are a 0.0 for every timestep except for the timesteps in which a line is cleared where the rewards are 1.0. The reward shaping used in this model includes a reward of -0.3 if the action taken causes a terminal state. The idea behind this reward shaping is that the model will learn that avoiding the termination of an episode gives higher rewards to the agent. The theory of

this reward shaping is that the model aims to make episodes last longer, and the longer an episode lasts the more likely it is for lines to be cleared. This, in turn, means that more positive rewards will be observed which means that there will be more positive samples for the model to learn towards [11].

4.2.3 Tetris results: Policy 1

Despite this model being trained over a short training period, the model does show empirical results of learning the Tetris environment. Given the sparseness of rewards, the performance of the policies is measured by the rate at which they gain positive rewards. This policy, over the training period, clears on average 1.4 lines every 100 episodes. Early in the training period, the policy shows a sufficient exploration strategy in which the rate of rewards gained is steep. In the videos of the agent’s gameplay, it can be seen that the agent does learn to spread out the Tetris pieces early on in an episode. It is in these early stages where most of the positive rewards are gained. This spreading out could be a consequence of the reward shaping that penalises early-ending episodes; the more spread out the Tetris pieces are, the lower the level of the pieces is, so the longer an episode lasts. The rate at which positive rewards are gained over training time varies, but even over a relatively short training period for the environment (models that learn Tetris typically learn over 10 million timesteps or more) the model shows promise for exploring the policy space.

4.2.4 Tetris results: Policy 2

Thompson sampling over the predictive distribution is seen to perform slightly worse compared to the first Thompson sampling policy. The average rate of lines cleared over the whole training loop is 1.2 lines every 100 episodes. This policy also shows a relatively steep incline to begin with, subsequently evening out later. A similar performance between these two policies is not too surprising as they work on the same principle of Thompson sampling, however this version does, on average, perform significantly worse. This may be because this version of Thompson sampling samples over a single value: the predicted q-value of a state-action pair, whereas the initial version samples over each weight value. This more extensive sampling could produce a more extensive exploration of the state space, and in a complex environment like Tetris, may be what is required to establish an effective policy.

4.2.5 Tetris results: Policy 3

A deterministic upper confidence bound shows very poor performance in the Tetris environment. Over the entire training loop, this policy only gains 12 positive rewards, averaging only 0.2 lines cleared per 100 episodes. An explanation for this can be easily found when observing the videos of the Tetris gameplay. In the videos, the policy demonstrates little exploration across actions and seems to repeat action choices very often. Despite the fact that this policy performs

well in a simple environment, such as Cartpole, this method of deterministic exploration, even with the utilisation of Bayesian statistics, doesn't perform well in the Tetris environment. This may be because the Cartpole environment is a simple environment and does not require too much exploration to be able to find an exploitative policy for it. Whereas to find an exploitative policy for the Tetris state space, a more detailed feature extraction needs to be established and, for this to occur, a more extensive exploration of the state space needs to be carried out. The deterministic UCB, however, chooses the optimum action at each stage, and despite the fact that the uncertainty is considered when choosing this action, a maximal selection such as this does not provoke enough exploration for an environment as complicated as Tetris.

4.2.6 Tetris results: Policy 4

The best performing policy was the stochastic upper confidence bound. As the deterministic upper confidence bound showed good performance in the Cartpole environment, it is surprising to see the severe lack of exploration the policy achieves in the Tetris environment. Therefore, a stochastic adaptation of the UCB can be made to enhance exploration. This adaptation is the forming of a distribution over the UCB sums performed by each action. This still makes the action with the highest upper confidence bound the most likely to be executed, but gives the other actions a chance at being selected as well. This adaptation not only vastly exceeds the performance of the deterministic UCB, but also performs better than the Thompson sampling policies. Over the training time, the stochastic UCB achieves 61 line clears, averaging a rate of 1.62 line clears every 100 episodes. As shown by the training curve, the rate at which lines are cleared seems to increase in the later stages of training. This shows that, with the exploration done in the initial stages, an exploitative strategy is established and when action selection relies on exploitation, the policy is sufficiently effective that lines clear at a faster rate and an increase in performance is observed.

4.2.7 Baseline Comparison

The initial Bayesian DQN is an adaptation from the Double DQN, therefore the baseline against which these models are compared is an ϵ -greedy Double DQN. This baseline model has a standard Q-network as a fixed target network and an online network. ϵ starts at 1 and decays over time where $\epsilon = 0.15$ at $t = 1,800,000$ and remains at this value for the remainder of training. The baseline agent also performs well, seeming to perform at a similar level to the initial Bayesian DQN and the stochastic UCB. Previous comparisons made between the baseline double DQN and the Bayesian DQN have shown the Bayesian DQN to outperform this baseline. Whether this can also be said about the Bayesian DQN is hard to say given the short training period these models and the baseline ran on. What can be concluded is that Thompson sampling and a stochastic UCB with a Bayesian linear regression head performs at a similar level to the ϵ -greedy Double DQN.

5 Further Work

This paper demonstrates the effectiveness that more principled exploration methods achieve in complex environments such as Tetris. It shows that, while defining the value of exploiting a particular action has obvious benefits, defining the value of exploring a particular action can also build a more effective policy. The models shown here take the more informative measures of the environment that Bayesian inference provides and, in different ways, formulate the values of taking specific actions that weigh up explorative value and exploitative value in an action. However, despite the positive results shown here, more work can be done on this study itself to extend the findings and can also be done on the broader concepts used.

5.1 Computation time

Whilst the models have demonstrated good performance over the training time used (2 million steps) RL models tested against Atari environments are generally tested over 10 million or more steps. This extended training period can give a model time to converge to an optimal policy and show more consistent performance. Therefore, the true potential of these models may have been better observed over an extended period of time, such as 10 million steps. As has also been shown in previous papers, in Atari environments the Bayesian DQN consistently exceeds the performance of the Double DQN. Over this short training period, these adaptations of the Bayesian DQN were only shown to perform similarly to the Double DQN, therefore a more comprehensive conclusion can be drawn if the training time of these models is increased and compared to the Double DQN over the same training period. For this extended training time, certain parameters, such as β_t for the UCB policy, will need to be reconfigured. Since the UCB performs well, at least in the stochastic case, a sensible configuration for the β_t parameter may be one that maintains the training time in a similar ratio of exploration and exploitation, as in this study.

5.2 Uncertainty as a measure for exploration

It has previously been stated how these models interpret the explorative value of certain actions. Whilst ϵ -greedy defines the explorative value of actions as, essentially, whether they are or are not the exploitative action, these methods define the explorative value in relation to the uncertainty of the value of an action. This exploration principle allows for a more dynamic and adaptive approach to exploration which will vary based on the observations made of the environment. This principle shows clear benefits over ϵ -greedy as the explorative value of different actions varies based on its uncertainty, where ϵ -greedy distributes the explorative values of actions evenly. It also bases it only on the fact that these actions are not the exploitative action, and also defines the trade-off between exploration and exploitation with a parameter that behaves independently of performance within the environment. However, even though

this is a more principled approach to exploration than ϵ -greedy, it is fair to ask whether uncertainty is an optimal measure for exploration. The aim of exploring in RL is to establish an exploitative policy that exceeds the current exploitative policy. Uncertainty only measures how unsure the value is about a particular action, not the potential for improvement in the value of an action compared to the value of the current optimal action.

An extended Bayesian method of RL that addresses this question of how to formulate exploration is Bayesian Optimisation. Bayesian Optimisation [8] is used to find maximum values of "black-box gradient-free functions that are expensive to evaluate". This method utilises Gaussian Processes, builds a "surrogate model" of the function and uses an "acquisition function" to guide the search for new points at which to attempt to find the optimum. This method has the exploitation-exploration trade-off inherent within it. One acquisition function used is similar to a method used in this paper: the upper confidence bound. Points are tried that maximise the sum of predicted value and weighted uncertainty. Another acquisition function is the expected improvement. This is the probability of a point yielding a more optimal value than the maximum value that has so far been observed. Therefore, promising potential for future work on this study is whether framing the exploration question as one which assesses the potential for improvement at a certain action is an overall better strategy for Tetris than merely questioning the uncertainty of certain actions.

6 Conclusion

In conclusion the methods in this paper exceed the performance of traditional RL methods for exploration, even over a short training period. This shows that for complex environments in which rewards are sparse, exploration needs to be principled and adaptive in order to find optimal policies. The more extensive, holistic view that Bayesian inference provides means that more creative and effective exploration strategies can be formed. These tactics explore a state space far more efficiently and enable optimal policies to be found faster. [9,510 words]

References

- [1] Kamyar Azizzadenesheli, Emma Brunskill, and Animashree Anandkumar. "Efficient Exploration through Bayesian Deep Q-Networks". In: *CoRR* abs/1802.04412 (2018). arXiv: 1802.04412. URL: <http://arxiv.org/abs/1802.04412>.
- [2] GeeksforGeeks. *Upper Confidence Bound Algorithm in Reinforcement Learning* [Image]. <https://www.geeksforgeeks.org/machine-learning/upper-confidence-bound-algorithm-in-reinforcement-learning/>. Image from the article "Upper Confidence Bound Algorithm in Reinforce-

ment Learning,” Last updated: 19 February 2020; accessed: 8 September 2025. Feb. 2020.

- [3] Mohammed Ghavamzadeh et al. “Convex Optimization: Algorithms and Complexity”. In: *Foundations and Trends® in Machine Learning* 8.5–6 (2015), pp. 359–483. ISSN: 1935-8245. DOI: 10.1561/22000000049. URL: <http://dx.doi.org/10.1561/22000000049>.
- [4] Emilie Kaufmann, Olivier Cappe, and Aurelien Garivier. “On Bayesian Upper Confidence Bounds for Bandit Problems”. In: *Proceedings of the Fifteenth International Conference on Artificial Intelligence and Statistics*. Ed. by Neil D. Lawrence and Mark Girolami. Vol. 22. Proceedings of Machine Learning Research. La Palma, Canary Islands: PMLR, 21–23 Apr 2012, pp. 592–600. URL: <https://proceedings.mlr.press/v22/kaufmann12.html>.
- [5] Pawel Ladosz et al. “Exploration in Deep Reinforcement Learning: A Survey”. In: *arXiv preprint arXiv:2205.00824* (2022). Available at: <https://arxiv.org/abs/2205.00824>.
- [6] Volodymyr Mnih et al. *Playing Atari with Deep Reinforcement Learning*. 2013. arXiv: 1312.5602 [cs.LG]. URL: <https://arxiv.org/abs/1312.5602>.
- [7] Keiron O’Shea and Ryan Nash. “An Introduction to Convolutional Neural Networks”. In: *arXiv preprint arXiv:1511.08458* (2015). Department affiliations: Aberystwyth University and Lancaster University.
- [8] Supratik Paul et al. “Robust Reinforcement Learning with Bayesian Optimisation and Quadrature”. In: *Journal of Machine Learning Research* 21.145 (2020), pp. 1–31. URL: <http://jmlr.org/papers/v21/18-339.html>.
- [9] Tom Rainforth. *Advanced Topics in Machine Learning: Bayesian Machine Learning*. <https://www.cs.ox.ac.uk/people/tom.rainforth/bml2020>. Lecture notes, Department of Computer Science, University of Oxford, Hilary Term. 2020.
- [10] Tom Schaul et al. *Prioritized Experience Replay*. 2016. arXiv: 1511.05952 [cs.LG]. URL: <https://arxiv.org/abs/1511.05952>.
- [11] SoftwarePatternsLexicon.com. *Reward Shaping: Improving Learning in Reinforcement Learning*. <https://softwarepatternslexicon.com/machine-learning/advanced-techniques/reinforcement-learning-strategies/reward-shaping/>. Accessed: 2025-09-08. July 2024.
- [12] Matt Stevens and Sabeek Pradhan. “Playing Tetris with Deep Reinforcement Learning”. Course project report, Stanford University. 2020. URL: https://cs230.stanford.edu/projects_winter_2020/reports/31834837.pdf.
- [13] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd ed. Second edition, in progress. Cambridge, Massachusetts; London, England: MIT Press, 2015.

- [14] Samandar Xamidov. *Cart Pole Gym using Reinforcement Learning [Image]*. <https://medium.com/@xamidovs005/cart-pole-gym-intodeep-q-learning-4d9315dac743>. Image from the article titled “Cart Pole Gym using Reinforcement Learning,” published February 16, 2023; accessed: 2025-09-08. Feb. 2023.